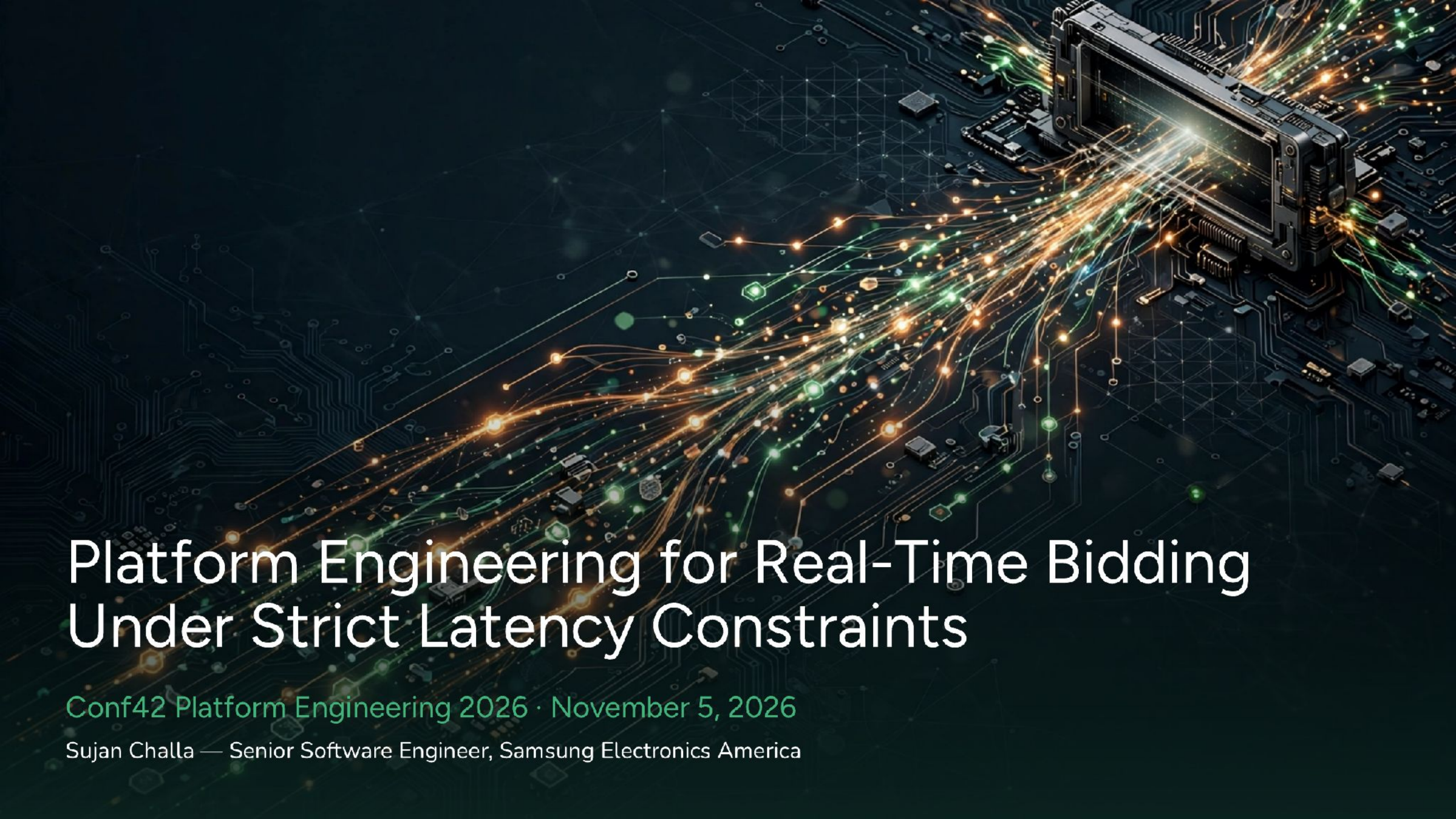


Platform Engineering for Real-Time Bidding Under Strict Latency Constraints

Conf42 Platform Engineering 2026 · November 5, 2026

Sujan Challa — Senior Software Engineer, Samsung Electronics America

My job turns business goals into reliable decision systems

Design

Translate revenue, latency, privacy, and product goals into service boundaries and data contracts.

Build

Go services, event pipelines, ML-serving paths, APIs, and CTV integrations.

Operate

Own latency, release safety, observability, incidents, and technical debt after launch.

Collaborate

Work across product, data science, ad operations, client teams, SRE, and partner engineers.

Production software is a loop, not a handoff

1

Frame the outcome

Product and engineering agree on the user value, constraints, and success metric.

2

Make decisions visible

Architecture reviews and design documents expose trade-offs before implementation.

3

Deliver in small increments

Backlogs, sprints, code review, automated tests, and integration environments reduce risk.

4

Release with guardrails

Canaries, metric gates, rollback paths, dashboards, and on-call feedback close the loop.

An entire market clears before the page feels loaded

1

An ad slot opens

A website or app offers one impression.

2

50+ systems decide

Each bidder values the context and chooses a price.

3

The auction clears

The highest eligible bid wins the slot.

4

100 ms

Request to response

The deadline makes the naïve design impossible

~100 ms

for the complete round trip

Network

The request and bid cross the network before useful work begins.

Consequence

A late answer is not a slower answer. It is no answer.

The first design



A database call for every request multiplies load and turns one slow dependency into the system's latency floor.

Speed beats perfect freshness on the live path



What becomes fast

User, context, campaign, and model inputs come from memory instead of a database round trip.

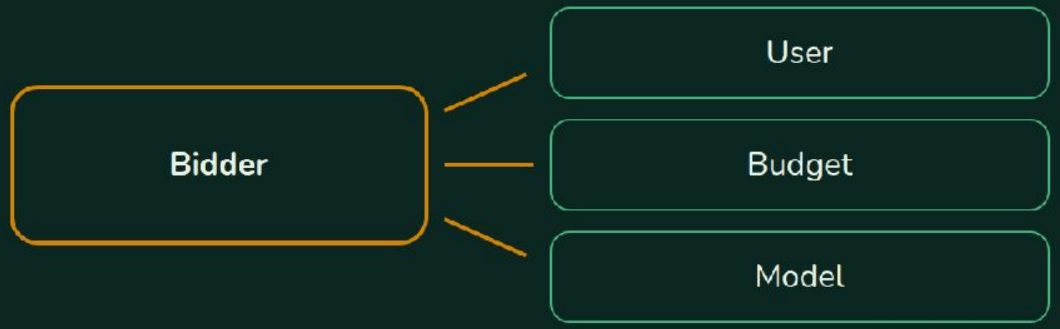
What becomes a trade-off

Data may be slightly stale, so expiration, invalidation, and source-of-truth recovery must be designed.

Architecture choice Reserve strict consistency for facts that truly require it.

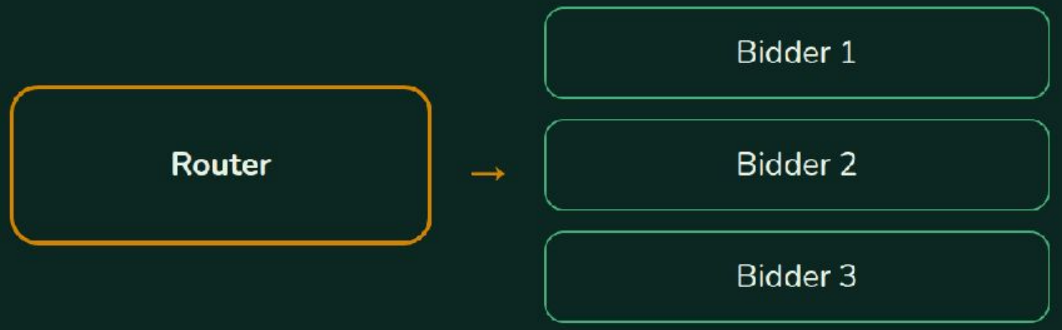
Parallel, stateless work preserves the budget

Parallelize independent waits



The slowest bounded lookup sets the join time—not the sum of every lookup.

Scale replicas, not state



Any healthy instance can serve any request; regions add capacity near demand.

CTV pods: many impressions, one decision



What the core can't see

A pod is several impressions that must be decided together. Two car ads back-to-back — or the same ad twice in one break — is a failure the per-impression auction can't even see.

What the edge layer owns

Assignment, deal tiers, competitive separation, and final assembly of the response. Cross-pod rules like frequency state ride shared state at this layer, not inside the auction.

Platform choice Don't fork the core. Decompose and reassemble at the edge — keep the core boring.

Pod-level rules: separation, frequency, deal priority

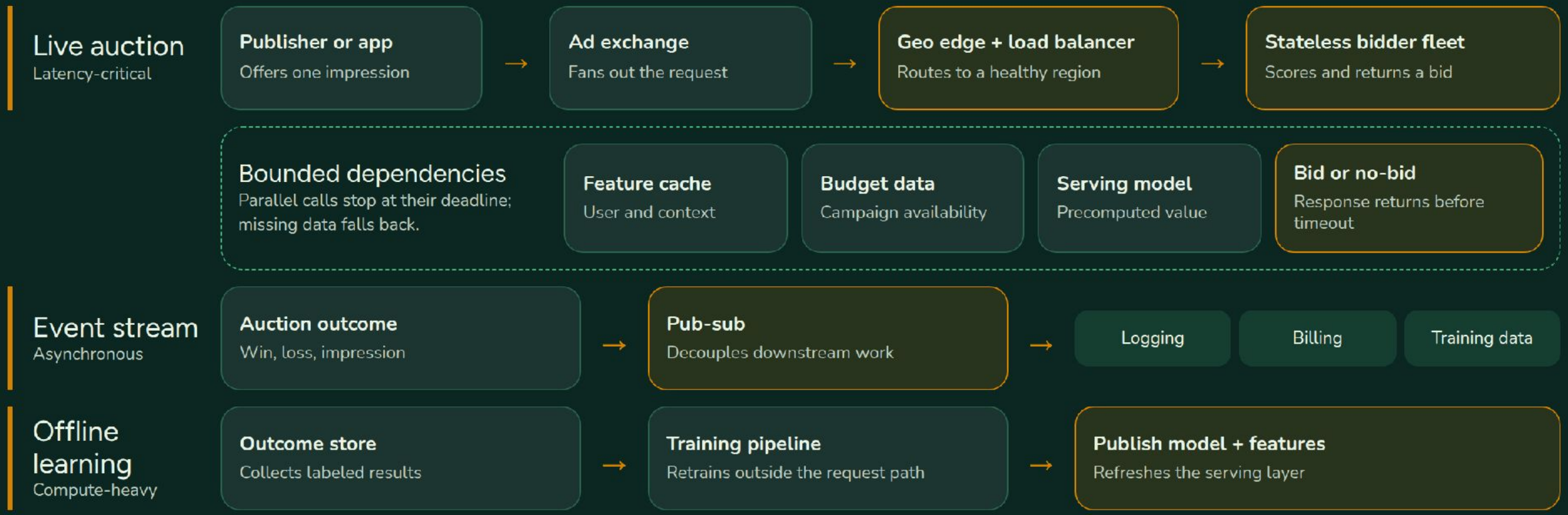
1. **Deal priority**
   Assignment fills the pod honoring deal tiers first — committed deals get what their tier promises, and the open auction competes for the positions that are left.

2. **Competitive separation**
   Enforced across the assembled pod. Two car ads in one break is a failure — however correct each individual auction was.

3. **Frequency control**
   Frequency state can't live inside a stateless per-impression core, so it rides shared state at the edge layer — checked and updated as each pod is assembled.

4. **The pattern**
   Any rule that spans impressions belongs at the layer that sees the whole pod. Push it into the core, and the core stops being boring — and stops being reusable.

The architecture separates the deadline from the learning loop



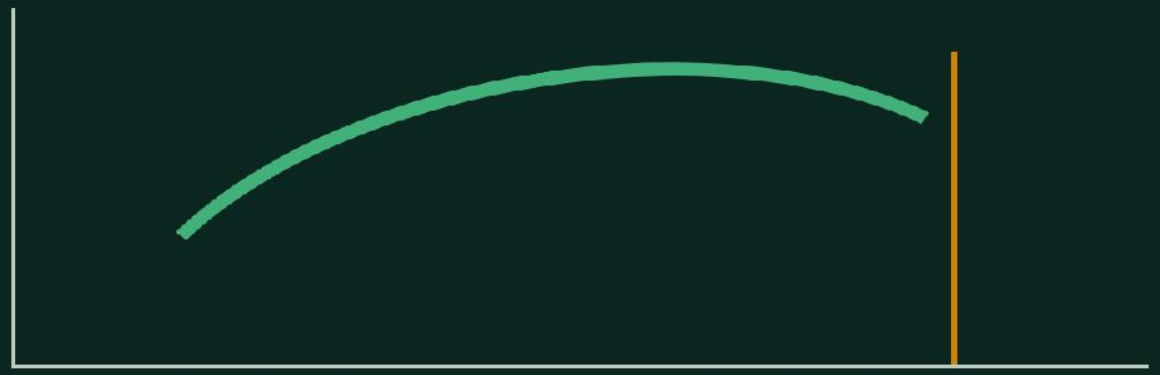
Every live-path dependency needs a deadline and an escape hatch

1. **Receive the request**
   Parse the impression and start one shared deadline.
2. **Fan out in parallel**
   Fetch user context, campaign budget, and model inputs together.
3. **Join what returned**
   Stop waiting when each sub-call reaches its timeout budget.
4. **Decide and respond**
   Use available signals to return a bid or no-bid before the exchange closes.

Failure	System behavior
Cache miss	Read the source, then repopulate the cache
Slow lookup	Use a sane default and continue
Repeated failures	Open the circuit and stop adding load
Deadline exhausted	Return no-bid rather than a late response

> A missing signal is cheaper than a missed auction.

Tail latency closes the engineering feedback loop



Watch p95 and p99

The average hides the requests most likely to miss the auction.

1. Publish auction outcome
2. Correlate metrics and traces
3. Train and validate offline
4. Refresh serving data safely

Production signals turn one release into the next architecture decision.

Lessons platform teams can steal

1. **Give every dependency a deadline and a default**
   Unbounded waits are how strict-budget systems die.

2. **Separate the deadline from the learning loop**
   If a request path waits on training, analytics, or reporting, the architecture — not the code — is the bug.

3. **Design degradation as a feature**
   Give it an owner, a metric, and a rehearsal — not an apology after an incident.

4. **Watch the tail, and budget your observability**
   p95, p99, p999 are the product metrics. Measurement itself spends the budget, so sample, aggregate at the edge, and ship asynchronously.

Thank you

Sujan Challa

Platform Engineering for Real-Time Bidding Under Strict Latency Constraints

Conf42 Platform Engineering 2026 · Find me in the conference chat, or connect on LinkedIn